Giovanni Capuano

Senior Software Engineer

Based in: **London**

Email: **hey@giovannicapuano.net**

About me



Trust, vision, meaning. With these values as my guiding principles, I'm motivated by a desire to **collaborate** closely with innovators to bring their vision to life. At Honeypot we helped families living in difficult parts of the world make their dreams come true, and at Simply Business, we took care of all the legwork so that our customers could focus on what really matters, safely.

I aim to bring more than just technical expertise to the table. I adopt an **holistic** approach to application development focusing on the **user experience, product workflows, and the overall business impact**. Additionally, I want to help teams to find the practices that work best for them allowing to **perfection development experience for delivering quality work** that is easy to **maintain and evolve**.

I am passionate in designing and developing **small, fast, language-agnostic, disposable, well tested and carefully documented** pieces of softwares and architectures, which behaviors and interactions are easily **traced, monitored and observable**. Ecosystems engineers are glad and incentivised to work with and extend, feel **ownership** of and are keen to **experiment and innovate**.

Ease of change, scalability, disaster recovery and keeping of decision records are topics I value to stress early on.

Professionally, in the latest couple of years I have worked extensively with tools and technologies such as

- **Docker Terraform GitHub Actions** and **Semaphore CI**
- **Ruby on Rails Grape Kotlin/Spring Boot** and **NodeJS**
- **TypeScript** and **React** (design systems, Storybook, feature flags and A/B experiments)
- **PostgreSQL** and **MongoDB** (change streams, Atlas)
- **Kafka Kafka Connect** (MongoDB and PostgreSQL source connectors) and **Redis**
- **AWS S3 SQS** and **ECS**

- **Pacts** and **OpenAPI**
- **CDC/Event schemas** and the **transactional outbox** pattern
- **Stripe** and **TokenEx** payment integrations
- **Amplitude** experiments, **Snowplow** and **GTM** analytics
- **OpenTelemetry NewRelic Backstage**
- **RBS/Steep** gradual typing for Ruby
- **Solidity** and **Smart contracts**

I started my path as an engineer back in 2012 when I wanted to build applications for supporting my friends and me building tactics and strategies for competitive Pokémon battles. That also led me to approach web development first with the a classic `LAMP` stack and then then with `Ruby` (`Sinatra`, `Hanami` and `Ruby on Rails`), `Kotlin` and the `Rust` programming language, which I started studying since its beta stage. I also enjoy building web-components with both modern `Vanilla JS` and `React`.

I also grew curiosity towards Arduino, Raspberry PI, Android and so on, interests that gave me reasons to look into languages such as `Swift` and practice a bit more `Java` and `C`.

In 2015 I accepted to join a digital studio company to meet a bunch of highly skilled professionals which gave me the chance to further **grow** as human being and engineer, and at the same time having the opportunity to **shape products** that had a real **impact** on people's life

From there I joined Honeypot, a startup that was in the process of going live, in which I contributed as second in-house engineer to the foundation of the culture and tech infrastructure. During that period I also contributed to the organization of the first edition of RustFest (https://rustfest.global) which had place in Berlin on 2016.

Currently, at Simply Business my team mates (a brilliant, cohesive and passionate bunch!) and I have gone from building the **event-streaming backbone** of the company to rebuilding the **customer-facing sales journey and checkout**, while constantly **expanding and sharing** our knowledge, with frequent and efficient **feedback and retrospectives** - and still I enjoy writing code as hobbist, in the attempt to create something **meaningful** for me and others, or just having fun by doing it.

Work experience

Simply Business

Senior Software Engineer

October, 2019 - Present

simplybusiness.com

Acquired by Travelers [NYSE: TRV] in 2017 and based in London and Boston, Simply Business is the **leading online business insurance brokerage**: it protects over one million small businesses across thousands of trades in the **UK and the US**, matching each of them with the cover suited to its risks and priorities.

I joined in 2019 as a Senior Software Engineer in the team that owns the company's **data streaming backbone** (Kafka, change data capture, domain events) and the back-office that handles **policy renewals and cancellations** for the UK and the US. Over the years I have taken projects **end to end**, from discovery and architecture decision records to production and aftercare, usually as the engineer who holds the thread while the team builds; argued for **unconventional technical choices** when they were the right ones and brought colleagues along with them; and looked after the things that let a team work well together: **documentation, observability, runbooks, pairing and facilitation**. Since late 2025 I have moved closer to the customer, rebuilding the **sales journey and checkout** in React and TypeScript.

Highlights

- Led the **TokenEx payment migration** to the new React architecture: built the first version and the patterns, then onboarded engineers from two teams so that we could finish it together (2026).
- Designed **Record Verification** and led its build in TypeScript: the service that proves the company's data lake matches production data, delivered on time for a finance audit (2025).
- With my team, took the **Kafka Outbox** from a team solution to the company standard for CDC and domain events; my part was migrating some fifty event types with the teams that owned them and extending it from MongoDB to PostgreSQL (2022 - 2025).
- Coordinated the move of **Rating Logs** out of the monolith into the rating service, with OpenAPI and Pact contracts and a clean teardown of the old infrastructure (2024 - 2025).
- Helped keep the **renewals application** modern since 2019: Rails 5.2 to 8, LDAP to SAML/Okta, Bootstrap 3 to 5, the first UI overhaul in about twelve years (2019 - present).
- Helped move the bulk of the company's data, the **quotesets**, from MongoDB to S3 with no code changes for the callers (2020 - 2021).

Projects

- **Sales journey frontend migration (late 2025 - present).** A strategic goal set by senior leadership: move the insurance sales journey from server-rendered Rails views to a **React 19 / TypeScript component architecture** built on the company's design system, one page at a time, behind feature flags and A/B experiments so that every step can be switched on, measured and rolled back independently.

  Each page has two halves and I work on both: the Rails page services, builders and typed contracts that feed the components, and the React components themselves, with their Storybook stories and tests. My pages were the **coverage selection, showcase and product recommendations** steps (trade imagery, coverage options, recommendation cards with estimated prices, an embedded insurance advisor widget) and the **purchase confirmation** pages for the UK and for US on-platform and off-platform purchases (policy details, account activation or sign-in, cross-sell, next steps, analytics datalayer), with Snowplow and GTM tracking aligned to the legacy pages so that funnel metrics stayed comparable throughout the migration.

  Ruby on Rails, React 19, TypeScript, Storybook, Amplitude experiments, Snowplow, GTM, RBS/Steep.

- **Checkout and payments (2026).** The payment step was the riskiest part of the migration. I **led the TokenEx part** (TokenEx is the card tokenisation provider): I built the first version and the patterns the rest of the work would follow, drew the flow as sequence diagrams so that everyone could reason about it, then onboarded engineers from two teams and coordinated the work until we had an HMAC-authenticated drawer checkout and a single-action payment flow. We shipped the React **Combined Payment Page** as the default for every customer and removed the legacy pages and the experiment that guarded them.

  Along the way I hardened the **Stripe** integration (idempotency keys against duplicate intents on refresh, cart digests to reject stale carts, prepaid card blocking) and built the payment and product summaries, the UK direct debit schedules and credit agreements, insurer-specific payment plans and the Amplitude experiment that decides which implementation a customer sees. Most recently I have been the engineer responsible for **multi-product carts**, from scoping to execution: a customer can buy several policies in a single checkout, with the payment combinations precomputed server-side and the summary adapting to the cart, rolled out behind a flag and measured in production.

  Ruby on Rails, React, TypeScript, Stripe, TokenEx, Amplitude, RBS/Steep.

- **Record Verification (2025).** For audit purposes the company had to **prove that the data landing in its data lake matches production**. We built a **Node.js / TypeScript service**, which I designed and led, that tails the MongoDB change streams of several databases in a cluster, batches and normalises the records and publishes their hashes to S3; the data platform hashes the same records once they land in Snowflake, and any mismatch, whether a record with different contents or a record present on one side only, raises an alert and ends up in a monthly audit report.

  I led it from the pre-project discovery and the spikes to production and aftercare, and **proposed TypeScript** over Ruby or Kafka Connect: an unusual choice for the company that I had to make a case for, and that the team came to enjoy and contribute to. We kept the pace a finance audit deadline demanded by splitting the work, some of us building components and others integrating them, and it went live on time in April 2025 for both the main monolith and the renewals application. Afterwards I paired with the data team on the edge cases and took the aftercare so that my teammates could start the next project without context switching, cutting the false alerts from both monitored applications. Resume tokens, heartbeats, versioned per-collection configuration, exact handling of money and nested fields, a recovery mode, OpenTelemetry tracing, New Relic dashboards, alert runbooks for the data team and the architecture decision record came with it.

  Node.js, TypeScript, MongoDB change streams, AWS S3, Snowflake, OpenTelemetry, New Relic, Airbrake, GitHub Actions, Semaphore CI.

- **Rating logs and upsells in the rating service (2024 - 2025).** Rating logs are the step-by-step audit trail of how a premium was calculated, used by pricing and partner-audit teams, and they lived in the monolith. I coordinated their extraction into the rating service: I ran the research and design conversations with the pricing, audit and rating teams, kept the story map and the cards current and made documentation part of refinement; on the code side I

built S3 and later MongoDB persistence, a versioned **OpenAPI** client gem, **Pact** contracts, CDC writes in a transaction, the back-office integration, the teardown of the old infrastructure (Terraform) and a custom RuboCop cop to stop the old model from creeping back.

Later we moved **upsells** into the rating service too: the add-on covers each insurer makes available are now computed at quote level in the rating pipeline and streamed through the schema registry and the CDC topics, so the journey no longer has to create and rate extra applications to price them. I instrumented the before and after on New Relic to demonstrate the impact. In the same period I contributed to the **temporary quotesets** capability, which lets the many flows that need a quote without intending to sell it (previews, comparisons, estimates) get one that is flagged, auto-deleted and kept out of events, CDC and rating logs, with a RuboCop guardrail so that nobody flags a real quote by mistake.

Ruby, Rails engines and gems, MongoDB, AWS S3, OpenAPI, Pact, Terraform, Kafka.

- **Kafka Outbox, the company-wide CDC and domain events platform (2022 - 2025).** Publishing to Kafka straight from the applications meant dual writes, messages lost under load or network errors, different failure rates for the database and for Kafka, and no way to observe or guarantee SLIs such as freshness and delay along the pipeline. We designed and delivered a **Change-Data-Capture solution based on the transactional outbox pattern**: applications write to an outbox in the same transaction as their data, a MongoDB Kafka source connector attached to a replica reads the change stream, a transformer service (Kotlin / Spring Boot) normalises the records and publishes them to read-view topics, and a sweeper confirms delivery and cleans up. Only data that successfully committed to the global and multi-region databases reaches consumers, with delivery guaranteed end to end. A sibling solution covered AWS S3 publishing through its notification system and SQS.

  From there the solution grew into the company standard, and my part was mostly the adoption: I coordinated and executed the Ruby and Mongoid upgrades that let every application in the organisation take it, migrated every domain event published by the main monolith (some fifty event types) and by the satellite services to the outbox with the teams that owned them, and decommissioned the legacy publishers; once Kafka Connect's single message transforms could do the normalisation, we retired the transformer and removed a hop from the pipeline. When another team adopted PostgreSQL I made the outbox library **ORM-agnostic** (MongoDB and PostgreSQL/ActiveRecord, Mongoid 8 and 9, Rails 7 and 8) with TTL retention, dead-letter recovery runbooks and a sweeper covering both stores, extended the Kafka source connector to PostgreSQL, added change-time headers and rebuilt its build/configure/deploy pipeline with circuit-breaker rollbacks. I also looked after the alerting (freshness SLOs on the transformer, dead-letter queue alerts as Terraform on New Relic), the dashboards, the team's 2025 **disaster recovery** review and the multi-region gamedays for the streaming stack, and wrote an organisation-wide RuboCop cop that forbids creating database indexes in the foreground.

  Ruby, Mongoid, ActiveRecord, Kafka, Kafka Connect, Kotlin/Spring Boot, schema registry, Terraform, New Relic, OpenTelemetry, AWS S3 and SQS.

- **Continuous Cover, the UK and US renewals and cancellations application (2019 - present).** The application renews expiring policies automatically where it can and gives consultants a dashboard for the rest and for US cancellations; before it, those workflows ran on large, complex and slow spreadsheets. We built it through research and development with fortnightly demos to a small group of users and stakeholders: few specifications stayed set in stone, and quick iterations on their feedback brought more happiness and efficiency to the teams.

  With the team I built the US workflow: renewable and paused policies listings with filters, sorting, pagination and bulk actions, CSV import with preview and export, notices of cancellation, carrier mail merge, moratoria, insurer management and the event consumers that keep the state in sync with the policy system. I have also been the one who keeps it current: Rails 5.2 to 8.0, Ruby 2.7 to 3.3, an unofficial epic to reach Mongoid 9, Sprockets to Shakapacker 8 and the first **Bootstrap** upgrade in about twelve years (3 to 5), which I proposed, split across the team and tested with a group of the renewals consultants, whose early feedback shaped the new screens. I replaced LDAP with **SAML/Okta single sign-on** together with the business operations team, researched and proposed SCIM for real-time user provisioning and walked the team through the whole epic. Unified structured logging, the IT audit and disaster recovery work of 2022 (health checks fit for autoscaling, the Kafka failover process, contingency runbooks), an operational status report and the one-off scripts and runbooks the operations team relies on complete the picture.

  Ruby on Rails, MongoDB, Sidekiq, Kafka, Bootstrap 5, Shakapacker, SAML/Okta, RSpec, Cucumber, New Relic.

- **Quoteset persistence on S3 (2020 - 2021).** The amount of data was approaching the hard limit of what the infrastructure could support. After analysing the needs of the different teams, we moved the bulk of it, the quotesets, to a **completely different kind of storage**: a Mongoid-compatible driver backed by S3, with caching and compressed serialisation, so that no caller needed a code change even though the data now lived elsewhere. It **saved the company a dramatic amount of money, made reads and writes faster** and went through without any disruption.

  Ruby, Mongoid, AWS S3, LocalStack.

Beyond the projects

- I revamped the way we **organise information** so that it is accessible, structured, collaborative and pleasant to work with, and I like to keep an archive of past states and digital ephemera (documents, emails, walkthroughs, diagrams, notes, briefs, incidents) that keeps answering questions years later: what I would call a **"librarian" culture**. In the same spirit I write and review **architecture decision records, major incident reviews and runbooks**, keep every service I touch documented on the internal developer portal and interconnect the documentation of the different projects (features, metrics, observability) so that engineers outside the team can find and use them.
- Day to day I **pair and explain a lot**, review pull requests quickly so that nobody waits on me, and take my turn on **incidents and alerts**. The feedback I value most is that colleagues learn

something each time we pair.
- I **facilitate retrospectives, risk storming and similar sessions**, designing each one around the context and the goals the project has at that moment rather than running a fixed format. I use them to try new techniques and ask the team for feedback afterwards, so that the formats get better over time.
- I built the **GDPR right-to-be-forgotten** tooling for policies and, later, for rating logs, so that a customer's data can be erased across the whole policy chain on request.
- When ownership moves, I hand it over properly: I transferred several business-support processes to other teams with written guides and **code walkthroughs** of the UK automated renewals.
- I work with **AI coding agents** every day and maintain the team's agent skills and guides for the component architecture, so that generated code fits the codebase and its conventions.
- I **interview engineers** applying to join the organisation.

Honeypot

Software Engineer

November, 2019 - May, 2018

honeypot.io

Honeypot is Europe's developer-focused job platform that let companies do the first move, using a powerful set of **search engine filters and keywords** and also **receving tailored emails** showing talents that are most relevant for them. Recruiters are also able to connect to their account **ATS** such as *Greenhouse* or *Personio* to avoid any additional manual work.

I joined the company as **second in-house full-time engineer** and during the 3 years and half (then it got acquired by the German LinkedIn competitor: XING/NewWork) I have contributed to enstablish a **developer-focused culture, shaped a solid and mantainable infrastructure, sane and safe workflows** and at every opportunity shared knowledge for providing a **clear understanding** of the software development processes among all the non-tech colleagues.

- I have done **daily round of interviews** to developers applying to join the organization and worked with my colleagues on establishing a sound internal hiring process.
- Supported the **Business Intelligence** team with the creation of a foundation of databases and workflows to allow them applying their algorithms and analysis on sandboxed, reliable data.
- Built an open source search engine allowing the search for talents becoming dozen of times faster, with the service being resilient and rock solid. I started working on it when Rust was not yet v1.0 and many features and libraries were not stable or fully-functioning yet - thus a lot of resources went to contributing to other projects (including the Rust buildpack for Heroku), to Rollbar and CircleCI and to the whole workflow to deploy and orchestrate all the different stages. Initial challenges aside, Rust proved to be a solid choice for such a kind of a project.
- Technologies I have been working with include: Ruby on Rails, Rust, Elixir, Ember.JS, Redis, Heroku, CircleCI and TravisCI, AWS Cloudfront and S3, Grape (RESTful APIs based on JWT

and TOTP authentication), Postmark and Mailchimp, Greenhouse and Personio, MindMatch AI, TalkJS...

- https://github.com/honeypotio/reina – a CLI tool and GitHub bot for solving our need of orchestrating the deployments of the feature stagings of our applications on Heroku
- https://github.com/honeypotio/searchspot – ElasticSearch-backed client/server written in Rust used by recruiters to search the talents they are looking for
- https://github.com/benashford/rs-es – Rust client for the ElasticSearch REST API I contributed to in order to make Searchspot
- https://github.com/RoxasShadow/rollbar-rs/ – library to track and report errors, exceptions and messages from Searchspot application to Rollbar
- Blog posts such as https://blog.honeypot.io/errors-and-exceptions-in-rust

Moze

Software Engineer

April, 2014 - October, 2015

mozestudio.com

Moze is an Italian studio of designers and developers which designs and builds digital products for entrepreneurs and forward-looking companies to shape or enhance digital products and services through a pragmatic design process and agile, incremental development.

I have been the first in-house full-time engineer after years the company relied mostly on contractors. I was close to every side of the projects, from the **front-end to the back-end to the dev-ops**.

- Among the other projects, we built Bugree: similarly to Proton Mail and MEGA, it was the first portal to make communications happens under NDA, and the first portal with **client-side encryption bounded to a dynamic key**, therefore messages never left any electronic footprint. There were more features like scrolling text, torch mode, limited openings and watermarks. Both messages and attachments were secured by end-to-end encryption and notifications were **real-time** thanks to a websockets-based NodeJS application we made. Its frontend was built with ReactJS and AngularJS and we implemented multiple payment processors including PayPal and Stripe.
- We also built GODO Shopping (for which we also made and open sourced zanox-rb, as we needed a good library to interact with the APIs of the popular affiliate marketing network service), which was a marketplace that with every purchase was granted a percentage in cashback to the buyer.
- Tourizan was another good project: like AirBnB but for enjoying unique experiences and places in Italy.

Bio

Born near **Naples, Italy**. Graduated in scientific high school and attended **Computer Engineering** at the University of Naples "Federico II" before leaving my hometown to accept a role at Moze, in **Milan**.

Italian, Neapolitan, English and some Spanish aside, I have been studying **Japanese** and joined an intensive course in a language school in Kyoto, in the meantime I was living with a local family thanks to the homestay program.

I love travelling and discovering new cuisines of all world but also all the regional cuisines of Italy. I love to prepare food and experiment new dishes everyday in my kitchen (and share the results on my Instagram!).

Being on holiday means for me means hiking mountains with friends and go discovering new pizza places or enjoying the seasonal menus of my favorites!

I'm a also passioned videogamer since I have memory of: always thirst of superb game and level designs, great soundtracks and meaningful stories.

Giovanni Capuano - hey@giovannicapuano.net - References on request